Perfeito, Douglas — agora vamos evoluir para um cenário mais completo e **mais próximo de um fluxo real de negócio (compra end-to-end)**, ainda respeitando rigorosamente as restrições:

- ✅ Usar Selenium diretamente
- ✅ Não usar Page Object
- ✅ Não usar Transaction
- ✅ Não criar novas fixtures
- ✅ Aproveitar driver existente

☑ 1. PLANO DETALHADO (PASSO A PASSO EXECUTÁVEL)

◇ FASE 1 — ENTENDIMENTO DO CENÁRIO

☑ Cenário completo

```
Compra com sucesso no SauceDemo
```

☑ Fluxo esperado

1. Login
2. Adicionar produto ao carrinho
3. Acessar carrinho
4. Iniciar checkout
5. Preencher dados
6. Finalizar compra

◇ FASE 2 — MAPEAMENTO DOS ELEMENTOS (INSPEÇÃO REAL)

Acesse: 👉 https://www.saucedemo.com

Abra DevTools (F12) e valide:

☑ LOGIN

Elemento	Selector
Usuário	id="user-name"
Senha	id="password"
Botão login	id="login-button"

☑ LISTAGEM DE PRODUTOS

Elemento	Selector
Botão adicionar produto	id="add-to-cart-sauce-labs-backpack"
Ícone carrinho	class="shopping_cart_link"

☑ CARRINHO

Elemento	Selector
Botão checkout	id="checkout"

☑ CHECKOUT - DADOS

Elemento	Selector
Nome	id="first-name"
Sobrenome	id="last-name"
CEP	id="postal-code"
Continuar	id="continue"

☑ FINALIZAÇÃO

Elemento	Selector
Botão finish	id="finish"
Mensagem sucesso	class="complete-header"

◇ FASE 3 — IMPLEMENTAÇÃO DO TESTE (DIRETO NO SPEC)

☑ Arquivo a editar

```
1  tests/specs/test_login.py
```

> 💡 (Você pode manter o nome ou criar test_checkout.py — recomendado para clareza)

☑ Código completo do teste

```
from selenium.webdriver.common.by import By

def test_compra_produto_com_sucesso(driver):

    # =========================
    # 1. LOGIN
    # =========================
    driver.get("https://www.saucedemo.com")

     driver.find_element(By.ID, "user-
name").send_keys("standard_user")
     driver.find_element(By.ID,
"password").send_keys("secret_sauce")
     driver.find_element(By.ID, "login-button").click()

     # Validação login
     assert "inventory.html" in driver.current_url

     # =========================
     # 2. ADICIONAR PRODUTO
     # =========================
     driver.find_element(By.ID, "add-to-cart-sauce-labs-
backpack").click()

     # =========================
     # 3. IR PARA CARRINHO
     # =========================
     driver.find_element(By.CLASS_NAME,
"shopping_cart_link").click()

     assert "cart.html" in driver.current_url

     # =========================
     # 4. INICIAR CHECKOUT
     # =========================
     driver.find_element(By.ID, "checkout").click()
```

```
33
34      assert "checkout-step-one.html" in driver.current_url
35
36      # =========================
37      # 5. PREENCHER DADOS
38      # =========================
39      driver.find_element(By.ID, "first-
name").send_keys("Douglas")
40      driver.find_element(By.ID, "last-name").send_keys("Teste")
41      driver.find_element(By.ID, "postal-
code").send_keys("12345")
42
43      driver.find_element(By.ID, "continue").click()
44
45      assert "checkout-step-two.html" in driver.current_url
46
47      # =========================
48      # 6. FINALIZAR COMPRA
49      # =========================
50      driver.find_element(By.ID, "finish").click()
51
52      # =========================
53      # 7. VALIDAÇÃO FINAL
54      # =========================
55      mensagem = driver.find_element(By.CLASS_NAME, "complete-
header")
56
57      assert mensagem.is_displayed()
58      assert "Thank you" in mensagem.text
```

◇ FASE 4 — EXECUÇÃO DO TESTE

☑ Rodar no terminal

```
1  pytest tests/specs -v
```

☑ Resultado esperado

```
PASSED
```

◇ FASE 5 — TRATAMENTO DE PROBLEMAS

☑ Problema: elemento não encontrado

✓ Verifique:

- ID mudou?
- Timing da página?

☑ Melhorar estabilidade (se necessário)

Adicionar espera explícita:

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.ID, "finish"))
)
```

◇ FASE 6 — EVOLUÇÃO IMEDIATA (ALTO VALOR)

☑ Criar novos testes:

1. Checkout sem preencher dados
2. CEP inválido
3. Remover item do carrinho
4. Comprar múltiplos produtos

☑ 2. SUMÁRIO (VISÃO DE ARQUITETO QA)

◇ Como o plano foi construído

✓ Baseado em fluxo real de usuário

- Login → Compra → Checkout → Confirmação

✓ Estratégia aplicada

Mesmo sem abstração:

- Mantém **legibilidade**
- Segue padrão **Given-When-Then**
- Garante **valor de negócio imediato**

✓ Decisões técnicas

Decisão	Motivo
Selenium direto	Restrição do cenário
Assert por URL	Validação rápida
Assert por elemento	Validação robusta
Fluxo único	E2E básico

☑ 3. BOAS PRÁTICAS (NÍVEL SÊNIOR)

◇ Estrutura do teste

Mesmo sem Page Object:

✓ Separe por blocos:

```
LOGIN
AÇÃO
VALIDAÇÃO
```

◇ Manutenibilidade

✅ Use comentários:

```
# 1. LOGIN
# 2. CHECKOUT
```

◇ Seletores

✅ Priorize:

- ID (já usado ✅)
- Classes estáveis

◇ Independência do teste

✅ Cada teste deve:

- Rodar isoladamente
- Não depender de outro

◇ Evitar flakiness

✅ Se instável:

- Usar waits explícitos
- Evitar time.sleep()

◇ Evolução Natural (PRÓXIMO PASSO)

Quando amadurecer:

✅ Refatorar para Page Object
✅ Criar Transactions
✅ Extrair dados para fixtures
✅ Criar suíte de regressão

🚀 CONCLUSÃO

Você agora tem:

✅ Um teste E2E completo de compra
✅ Representação real do negócio
✅ Base para regressão automatizada
✅ Implementação simples e direta

💡 **Visão de Arquiteto QA**

Esse teste já entrega alto valor porque:

- Valida fluxo crítico (receita)
- Reduz esforço manual
- Serve como base para regressão

Se quiser, posso te evoluir isso para nível avançado:

👉 Versão com **Page Object + Guará (padrão enterprise)**
👉 Suite completa do SauceDemo (10+ cenários)
👉 Pipeline CI com execução automática

Só me diga o próximo passo 👍